

Script Notes and Developer Log

C.J. Tinant

July 31, 2025

Contents

0.1	Overview	1
0.2	Project Notes (General)	1
0.3	TODO (General)	1
0.4	Future Ideas	1
0.5	Notes by Script	1
0.6	Zettelkasten-style markdown	5
0.7	Notes	6

0.1 Overview

This document collects implementation notes, design decisions, and development comments across scripts used in the regional skew estimation project. It supports ongoing improvements and serves as a lightweight developer log.

Use this log to track:

- Known issues or limitations
- Planned improvements (TODO)
- Script-specific decisions or reminders
- Ideas for refactoring or cleanup

0.2 Project Notes (General)

0.3 TODO (General)

0.4 Future Ideas

- Add `docs/variable_scaffold.csv` after covariate extraction to distinguish between exploratory output and final cleaned results.
- Adopt a targets-based workflow `{targets}` after covariate extraction.

0.5 Notes by Script

0.5.1 01a_download_ecoregions.R

- Original metadata & layer files for each level are downloaded for reference.

- Data sources are EPA/CEC shapefiles hosted via AWS links.
 - This script assumes internet access and local write permissions.
-

0.5.2 01b_download_USGS_gage_data.R

- Requires internet access to download data from USGS NWIS.
 - Bounding box grid helps avoid request size limitations in NWIS queries.
 - Uses a batch download approach for peak flow data retrieval.
 - Great Plains extent is defined using EPA Level 1 Ecoregion shapefiles.
-

0.5.3 01d_download_us_peakflow_data.R

- The script inputs data from 01c_download_usgs_gage_metadata.R
 - The script does not apply filtering on regulation, dam failure, or record length.
 - Filtering occurs in the subsequent script – 01e_filter_usgs_peakflow_data.R
-

0.5.4 01e_filter_usgs_peakflow_data.R

- Bulletin 17C explicitly recommends using the Multiple Grubbs-Beck Test (MGBT) to identify low-end outliers, e.g. zero flows, and near-zero flows like 0.01, 0.02, 0.05, etc. MGBT() flags observations below a calculated threshold discharge.
 - The inclusion of outliers (note in FFA there are only low-end outliers) distorts the estimation of the log-Pearson Type III distribution, biasing skewness, mean, and standard deviation. PILFs tend to skew the lower tail, by underestimating the sample skewness and producing unrealistic estimates for rare floods.
 - Peak codes are comma-separated and must be parsed for reliable flagging
 - Skewness is calculated as type = 1, using a method of moments estimator which follows the guidances under Bulletin 17C
 - Install peakfq directly from USGS GitLab `remotes::install_gitlab("water/peakfqr", host = "code.usgs.gov")`
-

0.5.5 01f_download_nhdplus_v2.R

- The code retrieves flowlines and catchments intersecting the buffered AOI
- `get_nhdplus()` loops through 144 tiles that intersect AOI.
 - It fetches data chunk-by-chunk and begins stitching them together.
 - Midway or After Completion: Invalid Geometry Detected
 - The function detects one or more invalid geometries (e.g., self-intersecting polygons or degenerate line segments).
 - It triggers a geometry repair step internally – sf / s2 Geometry Repair Mode Kicks In. This temporarily activates s2 geometry engine to fix topological errors, which is common in large hydrologic datasets. You see:
 - * Found invalid geometry, attempting to fix.
 - * Spherical geometry (s2) switched on
 - * Spherical geometry (s2) switched off
 - * Tiles Are downloaded again. The function starts over, reloading the same set of tiles (tiles 1–144) with geometry corrections in place.
 - This is expected behavior in nhdplusTools when:

- * An invalid feature is encountered (common with complex catchments or clipped geometries), and `get_nhdplus()` must retry with geometry repair enabled.
-

0.5.6 01g_download_nhdplus_hr.R

Key Features of the script: - Idempotent: skips previously downloaded regions - Geometry-safe: cast, buffer, validate for AOIs - Fuzzy retry logic: name correction and tile fallback - Transparent logging: CSV-based log with timestamped status - Interactive QA: mapview preview of results

0.5.7 01m_download_nlcd_2016.R

The NLCD raster was manually downloaded. Using `{FdData}` All tile requests timed out at ~30 seconds. Received over 100–200 MB per tile, but not enough to complete the download. Then `FdData::get_nlcd()`, tried to crop those partial files, which resulted in a crash.

The script inputs the following files and projects the raster to the vector CRS.

- a raster file: a `SpatRaster` (e.g., the full NLCD)
- a vector file: a `SpatVector` (e.g., Great Plains boundary)

Next, the script applies the following functions:

- `crop()` – Trims the raster extent down to the bounding box of the vector geometry (shape). Keeps all raster cells that intersect the box — even if they fall outside the exact shape.
 - `mask()` – Replaces all raster cells outside the exact shape with NA. Keeps only values within the actual polygon boundary.
-

0.5.8 01n_download_ned.R

Slope was calculated using (Fleming & Hoffer / Ritter algorithms), which use only the four cardinal directions (rook's case) to produce smoother, more generalized slope surfaces, which better matches subtle terrain transitions, especially in agricultural, prairie, or floodplain contexts.

When clipping I used `expand = 1000` for buffer to help prevent clipping artifacts near edges

0.5.9 01o_download_modis_ndvi_2016.R

NDVI, or Normalized Difference Vegetation Index is a measure of photosynthetic activity occurring within a pixel, which is commonly used to assess the health and density of vegetation. NDVI ranges from $[-1, +1]$, with healthy vegetation typically between $[0.6, 0.9]$. Higher NDVI values indicate healthier, more vigorous vegetation.

- $NDVI = (NIR - Red) / (NIR + Red)$

NDVI for the Great Plains ecoregion is calculated from MODIS, or Moderate Resolution Imaging Spectroradiometer data on the NASA Terra satellite. The relevant data, 250m resolution, 16-day composites, are labeled as MOD13Q1.

- MOD = MODIS sensor on Terra satellite
- 13 = Product suite 13: Vegetation Indices
- Q1 = 16-day temporal composite
- 061 = Collection 6.1 (latest operational version)

MOD13Q1 tiles contain multiple subdatasets (SDS): - NDVI — Normalized Difference Vegetation Index - EVI — Enhanced Vegetation Index - VI Quality — Bit-packed QA layer - Reflectance — Red, NIR, Blue bands used for index calculation - Day of Year — Date of composite

MODIS data are projected to a sinusoidal projection to preserve the relative size of land areas on the Earth's surface. The sinusoidal projection is an equal-area projection. MODIS tiles are 10-degrees by 10-degrees at the equator, and are named using Cartesian coordinates such that a tile named like h10v05 indicates the tile is the tenth tile from the origin in the horizontal direction and the fifth tile in the vertical direction from the origin (see https://modis-land.gsfc.nasa.gov/MODLAND_grid.html).

NDVI was chosen over EVI or Enhanced vegetation index for the Great Plains ecoregion. EVI is an index similar to NDVI that reduces the effects of: canopy background, aerosols (blue-band correction), and soil brightness, which often improves performance in dense forests. NDVI was selected for this project for the following reasons:

- Moderate to sparse vegetation: prairie, cropland, rangeland.
- Low cloud/aerosol interference: no need for blue-band correction.
- Wide use in agriculture and rangeland applications, such as crop condition, forage productivity, and drought/grazing assessment (VegDRI, USDM, NDMC).
- Fewer assumptions and post-processing corrections.

0.5.10 02a_merge_nhdplus_hr_flowlines.R

Type coercion is the process of converting a value or object from one data type to another. This can occur either automatically (implicitly) or intentionally (explicitly). Explicitly fixes issues using a file-reading loop with type coercion and error logging. The loop fixes the following issues: - Only coerce if possible, using a safe test like `is.integerish()`. - Fallback to numeric when needed. - Applies coercion one column at a time - Skips columns that cause errors (and logs them) - Prevents `across()` from failing all at once - Wrap the `across()` call in logic that filters to only existing columns. - Fine-grained control: logs problems per column, per file - Non-blocking: does not interrupt the whole region's read if a single column fails - Quiet warnings: keeps logs readable but still lets you know what happened - coerce fdate to character safely in all files. Character is the most flexible, readable, and safest for uncertain timestamp formats.

0.5.11 utils/process_geometries.R

Centroid locations, which the function extracts are useful for map labeling, visualization, and spatial summaries. The function applies strict checks for geometry validity and centroid coordinate extraction to avoid errors during plotting or labeling. The safeguards include checks for NA geometries, checks for empty or NULL centroids, explicit verification of coordinate matrix structure before extraction. Some additional details about `process_geometries()`:

Geometry Validation: `st_make_valid()` is used to correct any invalid geometries within the spatial data frame. The function identifies indices of valid geometries to ensure that centroids and subsequent operations are only applied to them.

Centroid Calculation: `st_centroid()` is applied conditionally only to valid geometries using `ifelse()`. If a geometry is invalid, NA is used as a fallback. `st_centroid()` is applied only to the valid parts of the geometry column. Centroids are stored in a list to handle any type of geometry being returned from `st_centroid()`. Each geometry is processed individually within a loop to allow more control over handling each item and better debugging capabilities if errors occur. `!is_empty(centroid)` checks if the centroid is not empty. The function also ensures that `st_coordinates(centroid)` actually returns a non-empty data frame before trying to access its elements.

Conditional Coordinates Extraction: The x and y coordinates are extracted from the centroid. If the centroid is NA (because the geometry was invalid), the coordinate fields are set to NA. Coordinates are

extracted only if there are valid centroids. This is safeguarded by checking if there are valid indices before attempting to extract coordinates. `text_x` and `text_y` are initialized with `NA_real_` to ensure that the type consistency is maintained for cases where centroids might not be computable. Before extracting coordinates, the function checks if the centroid is not NA and contains rows. Then it ensures that the coordinates can be indexed properly, and has the required number of columns (at least two, for x and y coordinates).

Additional checks: The function has additional checks for NAs and data structure validity. The function checks if centroid is not NA and not empty. Then, if coordinates are derived, the function ensures it is not NA and has the necessary rows and columns. The function avoids coercion errors by ensuring each part of the conditional is valid before evaluating the next part, this prevents logical operations on possibly undefined or inappropriate data types. The function logic is structured to progressively verify conditions before accessing potentially problematic attributes like the number of rows or columns. The function ensures that `coords` is not null before proceeding to check its dimensions. This prevents logical errors when `coords` might be an unexpected type or structure. The function more reliably ensures the data structure is correct before attempting to access its elements by using `is.null` along with checks for the number of rows and columns in `coords`

0.5.12 03a_update_covariate_metadata.R

Notes on project data structure: - Project datasets consist of feature data, vector data, and raster data stored in `~/data/processed/` - The response variable is station skew, which can be represented as vector point data. - The 63 initial explanatory variables, i.e. covariates or ‘covar’ in the script below, are structured by hierarchical scale (ordinal data) and domain (categorical data) - The covariate data are aggregated at each scale: Hierarchical Scale Aggregated by: 0 - Station NA 1 - Macroregional Macrozone 2 - Regional Level II Ecoregion 3 - Subregional Level III Ecoregion 4 - Local NHDPlusHD catchment

Domain Category A - Climate B - Land Cover C - Topography D - Watershed Metrics

0.6 Zettelkasten-style markdown

Zettelkasten-style markdown refers to using Markdown files to implement a digital version of the Zettelkasten method—a powerful personal knowledge management system developed by German sociologist Niklas Luhmann.

Zettelkasten means “slip box” in German. It’s a system where you:

- Break ideas into atomic notes (each note = one concept or insight)
- Use unique IDs and bi-directional links to connect notes
- Emphasize networked thinking rather than hierarchical folders

In R or code-based workflows, Zettelkasten-style Markdown means using plain text `.md` files where each file is a single, focused idea, tagged and cross-linked like a mini personal wiki.

0.6.1 Example Folder Structure

```
notes/  
  20250728_gam-vs-ridge.md  
  20250727_skewness-viz-notes.md  
  20250721_prism-uncertainty.md  
  index.md # optional index or dashboard
```

Each note might contain:

GAM vs Ridge Regression for Skew Modeling

- Ridge performed best on full covariate set (low MSE)
- GAMs improved interpretability in non-linear effects (e.g. `tmean_ann_C`)
- TODO: test GAM + macrozone as random effect

Linked notes:

- `[[20250721_prism-uncertainty]]`
- `[[20250727_skewness-viz-notes]]`

0.6.2 Core Features of Zettelkasten Markdown Notes

Feature	Benefit for Researchers
Atomic Notes	Each <code>.md</code> = 1 idea = easier reuse
Links (<code>[[...]]</code>)	Creates networked ideas, not silos
Timestamps / IDs	Keeps notes organized and unique
Emergent Structure	You discover themes organically
Markdown Format	Easy to version, edit, and render

0.6.3 How It Might Help

For your hydrologic and regional skew work, this approach could help:

- Track decisions or insights across milestones
- Store modeling rationale (e.g., why you dropped ecoregions)
- Connect field notes, exploratory thoughts, and literature summaries
- Gradually build a knowledge graph of ideas and results

ChatGPT can generate a Zettelkasten-style notes/ folder with a few seed notes and a script to open/edit them easily using RStudio.

0.7 Notes

- Last updated: 2025-07-31 12:01
- Maintained by: CJ Tinant